

Recap of Python Fundamentals – Practice Worksheet

16th May 2026 - Brendan McCann

Rather than teaching something new, we're going to go over Python fundamentals, just to make you guys as good as possible with the basics. We'll go over some topics last term, and we'll start with some easy exercises then make them slightly more difficult. Ask an educator if you need help - we literally get paid to help you guys!

Printing and Variables

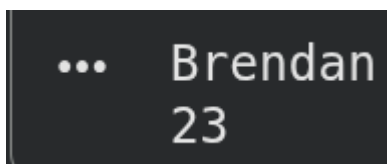
To print something to the screen, use:

```
print("Hello, World!")
```

That's pretty straightforward though. Let's introduce something called a **variable**. If you forgot, a **variable** is something that **stores information** - that could be a string (letters) or numbers, etc.. For example:

```
My_name = "Brendan"  
my_Age = "23"
```

Copy the 2 lines of code above, and try to print the values to the console. When you run your code, you should get something like:



```
... Brendan  
    23
```

Feel free to change the values of the variables to your own age and name! For example, if your name is Billy and you're 14, then do:

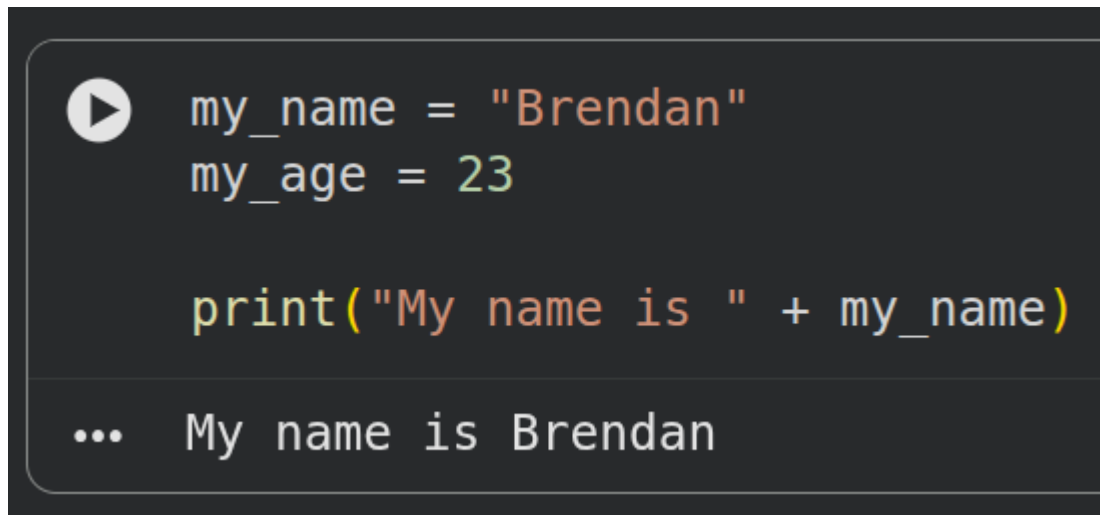
```
My_name = "Billy"  
My_age = "14"
```

If you need help, ask an educator.

Cool, let's do something a bit different. If you want to print some additional information to the screen, you can write code like:

```
print("My name is " + My_name)
```

Run your code - you should get something like:

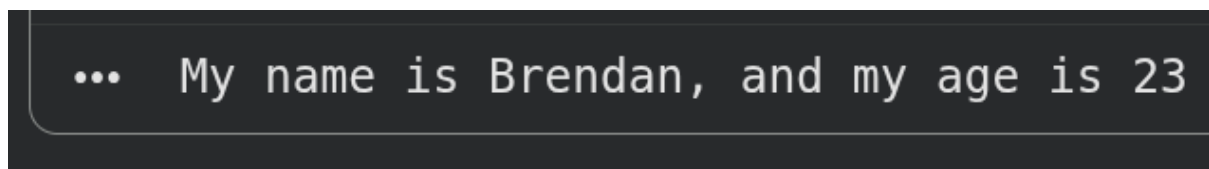
A screenshot of a code editor with a dark background. On the left, there is a play button icon. The code is written in a light-colored font. The first two lines are variable assignments: `my_name = "Brendan"` and `my_age = 23`. The third line is a print statement: `print("My name is " + my_name)`. Below the code, the output is shown: `... My name is Brendan`.

```
my_name = "Brendan"
my_age = 23

print("My name is " + my_name)

... My name is Brendan
```

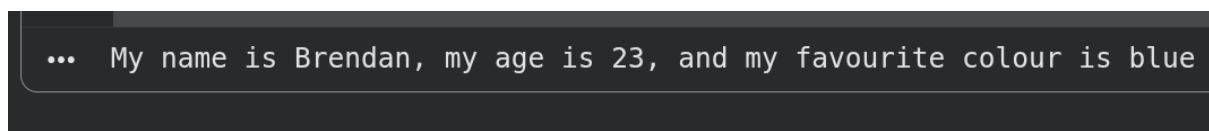
Ok, now try to write code to output something like this:

A screenshot of a code editor with a dark background. It shows the output of a print statement: `... My name is Brendan, and my age is 23`.

```
... My name is Brendan, and my age is 23
```

If you can't figure it out, ask an educator.

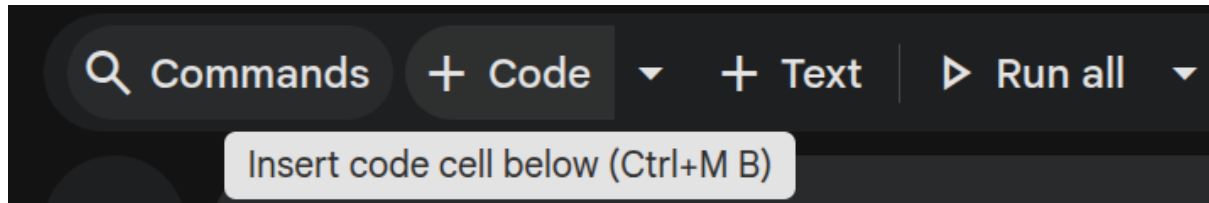
Now, create a variable called `my_favourite_colour`, and assign a value to it. Now, try to print out something like:

A screenshot of a code editor with a dark background. It shows the output of a print statement: `... My name is Brendan, my age is 23, and my favourite colour is blue`.

```
... My name is Brendan, my age is 23, and my favourite colour is blue
```

Taking user input

In this section, we're going to be working with different code, so press the '+ Code' button to create a new code cell. (look at the screenshot below):



When you run code, you can ask a user to type in something. For example, use the code below, and run it to see what happens:

```
User_name = input("What is your name? ")
```

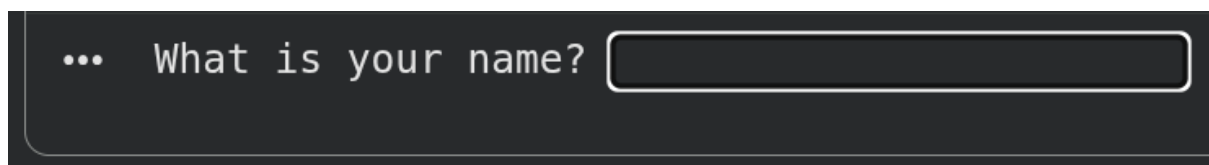
When you run the code, it will ask the user to type in something. Whatever the user types in, that value will be stored in the `User_name` variable.

To better understand what this means, run the code below:

```
User_name = input("What is your name? ")  
print(User_name)
```

Run the code and see what happens! If the result was not what you expected, ask an educator and we can explain it for you!

Cool, now let's try to create a little more of a sophisticated program. Create a program that first asks the user for their name, like this:



Then after typing in your name, it should print something like this:

```
... What is your name? Brendan
    Your name is Brendan
```

Ok, let's create something a bit more advanced. Try to create a program that first asks for the user's name:

```
... What is your name? 
```

Then it should ask for the user's age:

```
... What is your name? Brendan
    What is your age? 
```

And then it prints out:

```
... What is your name? Brendan
    What is your age? 23
    Your name is Brendan and your age is 23
```

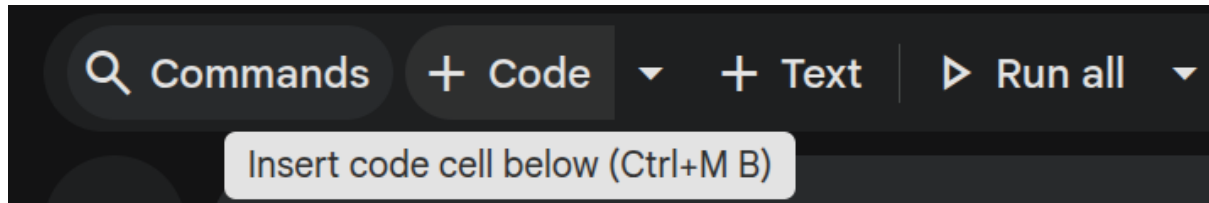
Do the same program again, but ask the user for their favourite colour. You should get an output that looks like:

```
... What is your name? Brendan
    What is your age? 23
    What is your favourite colour? Blue
    Your name is Brendan and your age is 23 and your favourite colour is Blue
```

Cool, if you got all of this done and it makes sense to you - you can move on to the next section! If not, ask an educator for help!

If-Else Statements

In this section, we're going to be working with different code, so press the '+ Code' button to create a new code cell. (look at the screenshot below):



Let's do a quick recap of if-else statements. Basically, this allows you to execute different parts of the code depending on some conditions. If that doesn't really make sense, it's fine because it will make more sense now!

```
My_age = 23

if My_age > 22:
    print("Unc")
elif My_age > 18:
    print("Yunc")
else:
    print("kiddo")
```

To best understand the code above, change around the values of the `My_age` variable: set it to 25, 19, and 16 and see what happens - don't be afraid to mess around with the code and tinker around! That's the best way of understanding things, especially in computer science and engineering!

To check if two values equal each other: you can use:

```
if My_age == 23:
    # ...
```

To check if two values are not equal to each other, use:

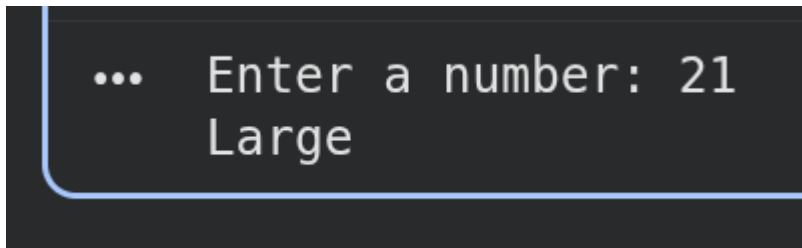
```
if My_age != 23:
    #...
```

Ok, let's do some exercises:

Ask a user to input a number by using:

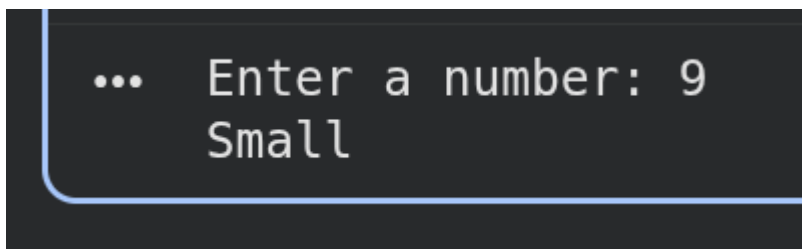
```
Number = int(input("Enter a number: "))
```

Then, create a simple program that prints "Large" if the number is greater than 10, otherwise the prints "Small". For example:

A terminal window with a dark background and light blue text. It shows a prompt '...' followed by the input 'Enter a number: 21' and the output 'Large' on the next line.

```
... Enter a number: 21
Large
```

And

A terminal window with a dark background and light blue text. It shows a prompt '...' followed by the input 'Enter a number: 9' and the output 'Small' on the next line.

```
... Enter a number: 9
Small
```

If you think you got that working, nice job! Let's do another exercise.

Create a variable with your name by using:

```
My_name = "PUT YOUR NAME HERE! FOR EXAMPLE, Brendan"
```

Then, ask the user for their name by using:

```
User_name = input("What is your name? ")
```

Create a simple program that prints out "Imposter!" if the User_name doesn't match My_name. If the names match, then the program should print "Welcome!". For example:

```
... What is your name? Brendan  
Welcome!
```

And

```
... What is your name? Billy  
Imposter!
```

Obviously the above example works for **my name (which is Brendan)**, and your code should work for **your name** (e.g. Alex, Bob, etc.)

Ok, let's do something a bit more advanced. To generate a random number between 1 and 10 in Python, use:

```
import random as rd  
  
random_number = rd.randint(1,10)
```

Create a program that will ask a user to guess a number between 1 and 10. If the user correctly guesses the random number, print "yay!", otherwise print "no :(". For example:

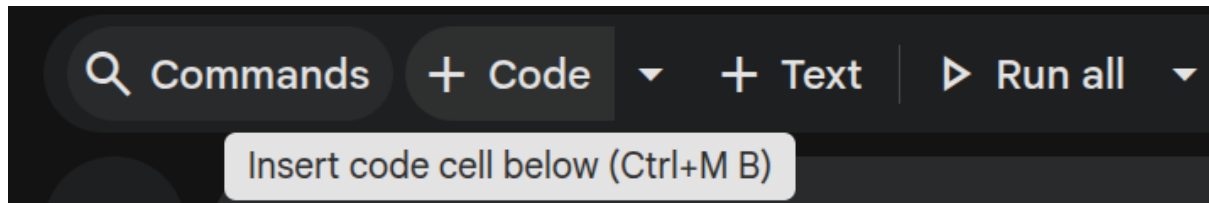
```
Guess a number between 1 and 10: 3  
no :(
```

Hint - use:

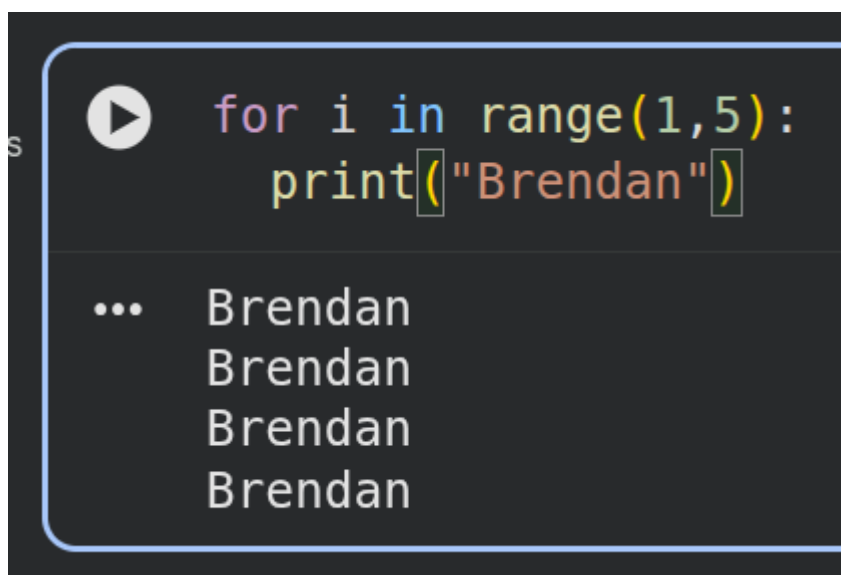
```
guess = int(input("Guess a number between 1 and 10: "))
```

For Loops

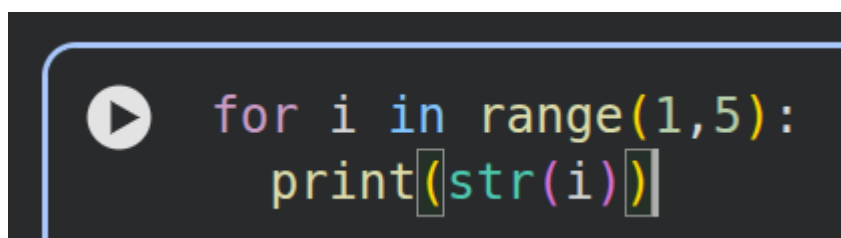
Ok cool, we are on to the final section! We're going to be working with different code, so press the '+ Code' button to create a new code cell. (look at the screenshot below):



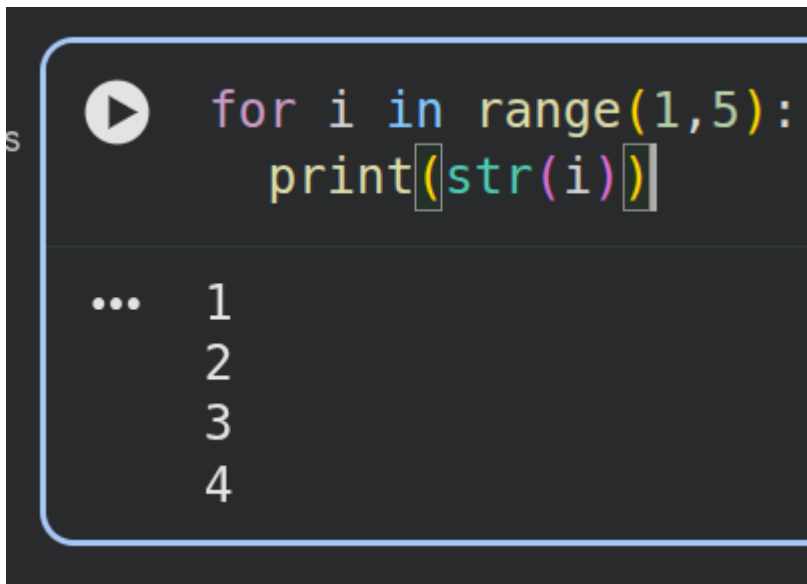
For loops allow you to repeat code in Python. For example, the code below will print out "Brendan" 4 times:



Before we move on, we have to understand the code above. The `range(1, 5)` generates a list of numbers from 1 up to - but not including - 5. To better understand what I mean, copy the code below and then run the program to see what the output is:

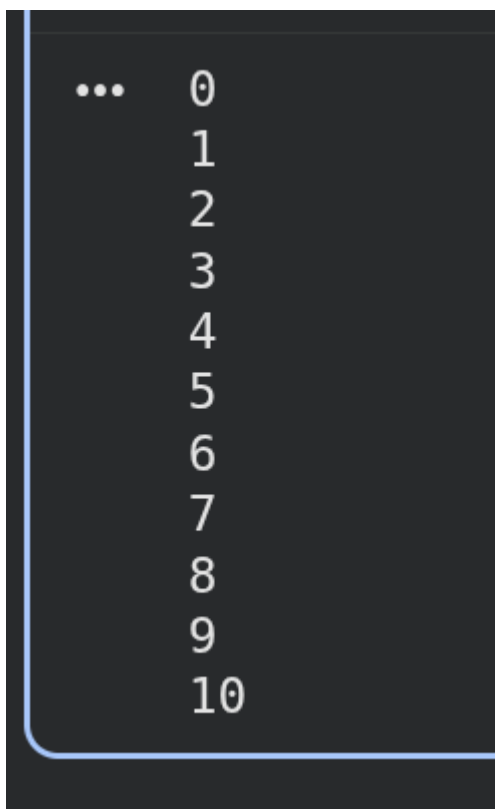


You should see the output looks like:

A screenshot of a Python code editor with a dark background. The code `for i in range(1,5):` is on the first line, and `print(str(i))` is on the second line. A play button icon is to the left of the first line. Below the code, the output shows three dots followed by the numbers 1, 2, 3, and 4, each on a new line.

```
for i in range(1,5):  
    print(str(i))  
  
... 1  
     2  
     3  
     4
```

Now, try to print out all the numbers from 0 to 10, so your program should look like:

A screenshot of a Python code editor with a dark background. The output shows three dots followed by the numbers 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, and 10, each on a new line.

```
... 0  
    1  
    2  
    3  
    4  
    5  
    6  
    7  
    8  
    9  
   10
```

Now try to print out the numbers from 5 to 11, it should look like:

```
... 5
      6
      7
      8
      9
     10
     11
```

Ok, just so you understand, a for loop in python executes codes multiple times. In this case, we are printing out the i variable multiple times.

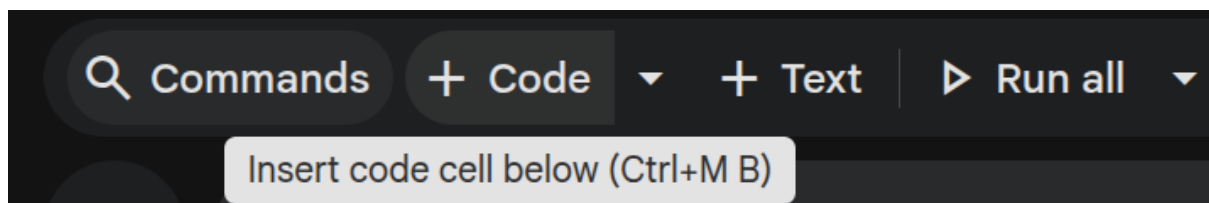
Now, let's combine our knowledge here. Create a program that asks a user for their name **five times**. For example:

```
... what is your name: brendan
    what is your name: brendan
    what is your name: brendan
    what is your name: brendan
    what is your name: brendan
```

Now, when you ask the user for their name, print their name out. For example:

```
... what is your name: brendan  
brendan  
what is your name: brendan  
brendan  
what is your name: brendan  
brendan  
what is your name: brendan  
brendan  
what is your name: brendan  
brendan
```

We're going to be working with different code, so press the '+ Code' button to create a new code cell. (look at the screenshot below):



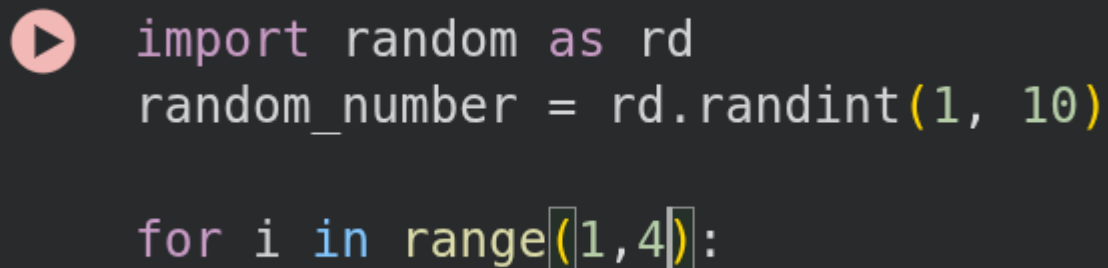
Generate a random number between 1 and 10 - use the code below:

```
import random as rd  
  
random_number = rd.randint(1, 10)
```

Now, create a for loop that iterates **three times** by using:

```
for i in range(1,4):
```

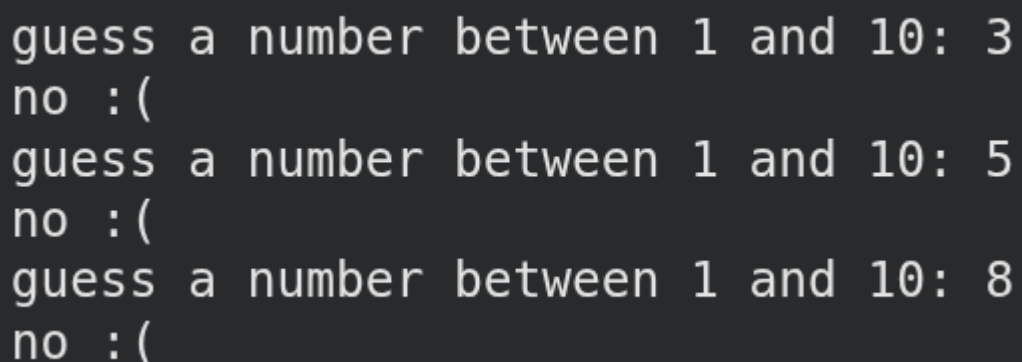
So, your code should look like:

A dark-themed code editor snippet with a pink play button icon on the left. The code is as follows:

```
import random as rd
random_number = rd.randint(1, 10)

for i in range(1,4):
```

Using the code we just gave you (as a starting point), create a random number guessing game! The user has three tries to guess a random number between 1 and 10. If the user guesses the number correctly, you should print “yay!”, otherwise print “no, please try again”. For example:

A dark-themed terminal window showing the output of a random number guessing game. The output is as follows:

```
guess a number between 1 and 10: 3
no :(
guess a number between 1 and 10: 5
no :(
guess a number between 1 and 10: 8
no :(
```